

Lekce 12: Konfigurační soubory Cypress a proměnné prostředí

Cíle

- Pochopit strukturu a roli konfiguračních souborů Cypress.
- Naučit se nastavovat globální konfigurace (např. baseUrl, nastavení viewportu) pomocí souboru `cypress.config.js`.
- Přizpůsobit chování Cypress pomocí nastavení konfigurací.
- Využívat proměnné prostředí pro správu citlivých dat a přepínání mezi různými prostředími (vývojové, testovací, produkční).
- Přistupovat k proměnným prostředí v rámci testů pomocí `Cypress.env()`.
- Zavést osvědčené postupy pro přehlednost, udržovatelnost a bezpečnost konfigurace.

Rozpis témat

A. Konfigurační soubory Cypress

1. Přehled o souboru `cypress.config.js`:

- **Definice:**

Soubor `cypress.config.js` (nebo `cypress.config.ts` při použití TypeScriptu) je centrálním místem, kde definujete globální nastavení pro vaše Cypress testy. V nových verzích nahrazuje starší formát `cypress.json`.

- **Účel:**

- Konfiguruje, jak Cypress spouští testy, včetně nastavení výchozích hodnot.
- Definuje vlastnosti jako `baseUrl`, rozměry viewportu, timeouty a další.

- **Struktura:**

Typicky exportuje konfigurační objekt. Například:

```
const { defineConfig } = require('cypress');

module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    viewportWidth: 1280,
    viewportHeight: 720,
    defaultCommandTimeout: 8000,
    supportFile: 'cypress/support/index.js',
    // Další konfigurační možnosti...
  },
  env: {
    // Proměnné prostředí lze nastavit i zde
    apiUrl: 'https://api.example.com'
  }
});
```

2. Nastavení globálních konfigurací:

- **baseUrl:**

- Nastavuje výchozí kořenovou URL adresu, abyste mohli v testech používat relativní cesty.

`baseUrl` je globální konfigurační nastavení v Cypress, které definuje výchozí URL pro vaši testovanou aplikaci. Tato hodnota je používána jako základ pro všechny relativní cesty v testech.

Účel a výhody:

- **Zjednodušení navigace:**

Místo zadávání celé URL u každého volání `cy.visit()` můžete použít relativní cestu. Pokud je například `baseUrl` nastaveno na `http://localhost:3000`, pak `cy.visit('/login')` se automaticky rozvine na `http://localhost:3000/login`.

- **Snadnější přepínání prostředí:**

Centralizací URL lze rychle přepínat mezi vývojovým, testovacím a produkčním prostředím bez nutnosti upravovat jednotlivé testy.

- **Čistší testovací kód:**

Snižuje opakování a potenciální chyby tím, že je pouze jedno místo s nastavením základní URL.

Příklad:

```
// cypress.config.js
const { defineConfig } = require('cypress');

module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    // ...další nastavení
  }
});
```

- **Nastavení viewportu:**

- Definuje rozměry okna prohlížeče během testování.
- **Příklad:**

```
viewportWidth: 1280,
viewportHeight: 720
```

- **Timeouty:**

- Nastavuje timeouty pro příkazy, odpovědi a další časově závislá nastavení.
- **Příklad:**

```
defaultCommandTimeout: 8000,
pageLoadTimeout: 60000
```

- **Vlastnost watchForFileChanges**

Definice:

`watchForFileChanges` je konfigurační vlastnost, která určuje, zda má Cypress sledovat změny ve vašich projektových souborech a automaticky znovu spouštět testy při zjištění změny.

Účel a výhody:

- **Efektivita vývoje:**
Umožňuje rychlejší vývojový cyklus automatickým spouštěním testů při každé úpravě kódu nebo testů.
- **Automatické obnovení:**
Dělá Test Runner interaktivnější a rychleji reaguje na změny v testovacím kódu.

Příklad použití:

```

module.exports = defineConfig({
  e2e: {
    watchForFileChanges: true, // Automaticky znovu spustí testy při změně souborů
    // ...další nastavení
  }
});

```

Poznámka: V CI prostředí můžete nastavit `watchForFileChanges` na `false`, abyste zabránili zbytečnému opakovanému spouštění.

- **Vlastnost retries**

Vlastnost `retries` určuje, kolikrát by měl Cypress automaticky zkoušet znovu neúspěšný test před jeho finálním označením za neúspěšný. Toto lze nastavit globálně nebo pro každý test samostatně.

Účel a výhody:

- **Zvládnutí nestabilních testů:**

Pomáhá zvládnout potíže s asynchronním chováním nebo časováním, zvláště v komplexních nebo nestabilních prostředích.

- **Zvýšená robustnost testů:**

Opakováním testů snižujete počet „falešných negativních“ způsobených přechodnými problémy.

Příklad použití:

```

module.exports = defineConfig({
  e2e: {
    retries: {
      runMode: 2, // Počet opakování v headless režimu (CI)
      openMode: 0 // Bez opakování v interaktivním režimu
    },
    // ...další nastavení
  }
});

```

V testovacím souboru můžete opakování nastavit i pro konkrétní skupinu:

```

describe('Login Tests', { retries: { runMode: 2, openMode: 1 } }, () => {
  it('should log in successfully with valid credentials', () => {
    // Test kód zde...
  });
});

```

- **Další globální nastavení:**

- `video` , `screenshotOnRunFailure` atd. pro ovládání záznamů během běhu testů.
- Zadání `supportFile` pro globální konfiguraci a vlastní příkazy.

3. Přizpůsobení chování Cypress přes konfigurační soubory:

- Upravte nebo rozšiřte výchozí chování Cypress pomocí pluginů a rozšířených nastavení.
- Využijte adresář `plugins` společně se `cypress.config.js` pro rozšíření možností.

- **Příklad:**

Integrace pluginu pro úpravu chování sítě nebo správu stahování souborů.

4. "e2e": {} a "env": {}

Sekce "e2e": {} :

- **Účel:**

Sekce "e2e" v konfiguraci Cypress slouží k definování nastavení a chování specifických pro end-to-end (E2E) testy. Obsahuje vlastnosti jako `baseUrl` , nastavení viewportu, výchozí timeouty a hooky pro nastavování Node událostí.

- **Použití:**

Tato sekce shrnuje všechna nastavení související s tím, jak Cypress bude spouštět vaše E2E testy.

Příklad:

```
module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    viewportWidth: 1280,
    viewportHeight: 720,
    defaultCommandTimeout: 8000,
    setupNodeEvents(on, config) {
      // Vlastní úlohy a pluginy
      return config;
    }
  }
});
```

Sekce "env": {} :

- **Účel:**

Vlastnost "env" slouží k definování proměnných prostředí, ke kterým lze přistupovat během běhu testu. Tyto proměnné pomáhají spravovat citlivá data (např. API klíče) a konfigurace, které se mohou měnit v různých prostředích (např. vývoj, staging, produkce).

- **Použití:**

Přistupujte k proměnným prostředí v testech pomocí `Cypress.env('variableName')` .

Příklad:

```
module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000'
  },
  env: {
    apiUrl: 'https://api.dev.example.com',
    userRole: 'admin'
  }
});
```

V testu:

```
describe('API Tests', () => {
  it('should fetch data from the correct API endpoint', () => {
    const apiUrl = Cypress.env('apiUrl');
    cy.request(`${apiUrl}/users`).then((response) => {
      expect(response.status).to.eq(200);
    });
  });
});
```

B. Proměnné prostředí v Cypress

1. Použití proměnných prostředí pro citlivá data:

- **Definice:**

Proměnné prostředí umožňují externalizovat konfigurační údaje, jako jsou API klíče, URL adresy a přihlašovací údaje. Díky tomu nejsou citlivá data přímo v testovacím kódu.

- **Použití v cypress.config.js :**

```
env: {
  apiUrl: 'https://api.example.com',
  authToken: 'your-secret-token'
}
```

- **Proč je to důležité:**

- Zvyšuje bezpečnost, protože citlivé hodnoty nejsou napevno v kódu.
- Umožňuje snadnou změnu hodnot bez modifikace testovacích souborů.

2. Správa různých prostředí (vývoj, staging, produkce):

- **Postup:**

- Používejte samostatné konfigurační soubory nebo nastavení proměnných prostředí pro každé prostředí.
- Předávejte proměnné prostředí přes příkazovou řádku při spouštění Cypress.

- **Příklad:**

```
npx cypress run --env config=staging
```

- **V konfiguračním souboru:**

Podle příznaku prostředí načtěte různé proměnné.

```
const config = process.env.CONFIG || 'development';

module.exports = defineConfig({
  e2e: {
    baseUrl: config === 'production' ? 'https://prod.example.com' : 'http://localhost:3000',
    // Další nastavení...
  },
  env: {
    apiUrl: config === 'production' ? 'https://api.prod.example.com' : 'https://api.dev.example.com',
  }
});
```

- **Lepší přístup k oddělení dat a konfigurace podle prostředí**

Ve složce cypress/config máme různé konfigurační soubory pro každé prostředí:

- cypress.test2.json
- cypress.production.json

Ve složce cypress/fixtures máme různé testovací datové soubory pro každé prostředí:

- data.test2.json
- data.production.json

```

import { defineConfig } from 'cypress'
import { rmdir, readFileSync } from 'fs'

export default defineConfig({
  e2e: {
    setupNodeEvents(on, config) {

      const envName = config.env.name;
      const envconfig = readFileSync(`./cypress/config/cypress.${envName}.json`, 'utf
      const configValues = JSON.parse(envconfig);

      const envdata = readFileSync(`./cypress/fixtures/data.${envName}.json`, 'utf-8'
      const dataValues = JSON.parse(envdata);

      config.env = { ...configValues, ...dataValues };
      return config;
    },
  },
})

{
  "scripts": {
    "open:test2": "npx cypress open --env name=test2",
    "test:test2": "npx cypress run --browser chrome --env name=test2",
    "test:test2:record": "npx cypress run --browser chrome --env name=test2 --record",
    "test:test2:ci": "pnpm test:test2:record",
    "open:production": "npx cypress open --env name=production",
    "test:production": "npx cypress run --browser chrome --env name=production",
    "test:production:record": "npx cypress run --browser chrome --env name=production -
    "test:production:ci": "pnpm test:production:record"
  },
}

```

3. Přístup k proměnným prostředí v testech:

- K proměnným definovaným v konfiguraci přistupujete pomocí metody `Cypress.env()`.
- **Příklad:**


```
describe('API Tests', () => {
  it('should fetch data from the API', () => {
    const apiUrl = Cypress.env('apiUrl');
    cy.request(`${apiUrl}/users`).then((response) => {
      expect(response.status).to.eq(200);
    });
  });
});
```

Použití setupNodeEvents v konfiguraci Cypress (cypress.config.js)

Co je setupNodeEvents ?

- **Definice:**

Funkce `setupNodeEvents` v souboru `cypress.config.js` umožňuje registrovat posluchače událostí, rozšiřovat chování Cypress nebo provádět úlohy, které vyžadují běh v Node.js.

- **Účel:**

- Rozšíření chování Cypress během běhu testů.
- Zpracování souborů, interakce s databází, požadavky na API nebo externí logování.
- Definice **vlastních úloh Cypress** umožňujících komunikaci mezi testovacím kódem (v prohlížeči) a Node.js (na pozadí).

Příklad: setupNodeEvents s vlastní logovací úlohou

Tento příklad definuje úlohu, která během běhu testů vypisuje zprávy do konzole.

cypress.config.js

```
const { defineConfig } = require('cypress');

module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    viewportWidth: 1280,
    viewportHeight: 720,
    defaultCommandTimeout: 8000,

    setupNodeEvents(on, config) {
      // Přidání vlastní úlohy pro logování zpráv z prohlížeče do terminálu
      on('task', {
        logMessage(message) {
          console.log('LOG FROM CYPRESS:', message);
          return null; // Úlohy musí vracet hodnotu nebo null
        }
      });

      return config; // Důležité vrátit konfigurační objekt
    }
  }
});
```

Použití v testovacím souboru (logování do terminálu)

```
describe('Logging Example', () => {
  it('should log a message from Cypress to the terminal', () => {
    cy.task('logMessage', 'This is a custom log from Cypress');
  });
});
```

Očekávaný výstup v terminálu:

```
LOG FROM CYPRESS: This is a custom log from Cypress
```

Pochopení `on('task')` v Cypress

Co je `on('task')` ?

- Funkce `on('task', { ... })` v `setupNodeEvents` umožňuje Cypressu **spouštět Node.js kód během testů**.
- Úlohy mohou sloužit k:
 - **Práci se soubory** (čtení/zápis souborů)
 - **Dotazům na databázi** (např. interakce s MongoDB, PostgreSQL)
 - **API interakcím** (volání externích API)
 - **Logování** (posílání logů do terminálu)

Praktické příklady použití `on('task')`

1. Zápis do souboru

Může být potřeba ukládat výsledky testů nebo logy do souboru.

cypress.config.js

```
const fs = require('fs');

module.exports = defineConfig({
  e2e: {
    setupNodeEvents(on, config) {
      on('task', {
        writeFile({ filename, content }) {
          fs.writeFileSync(filename, content);
          return null;
        }
      });

      return config;
    }
  }
});
```

Použití v testu

```
describe('Write File Example', () => {
  it('should write test data to a file', () => {
    cy.task('writeFile', {
      filename: 'cypress/logs/test-output.txt',
      content: 'Test executed successfully at ' + new Date()
    });
  });
});
```

2. Čtení souboru

Pokud potřebujete ověřit obsah souboru.

cypress.config.js

```
module.exports = defineConfig({
  e2e: {
    setupNodeEvents(on, config) {
      on('task', {
        readFile({ filename }) {
          return fs.readFileSync(filename, 'utf8');
        }
      });

      return config;
    }
  }
});
```

Použití v testu

```
describe('Read File Example', () => {
  it('should read content from a file', () => {
    cy.task('readFile', { filename: 'cypress/logs/test-output.txt' }).then((content) => {
      expect(content).to.contain('Test executed successfully');
    });
  });
});
```

3. Generování náhodných dat pomocí Faker.js

Pokud chcete vygenerovat náhodná data pro registraci uživatele.

cypress.config.js

```
const { faker } = require('@faker-js/faker');

module.exports = defineConfig({
  e2e: {
    setupNodeEvents(on, config) {
      on('task', {
        generateRandomUser() {
          return {
            username: faker.internet.userName(),
            email: faker.internet.email(),
            password: faker.internet.password()
          };
        }
      });

      return config;
    }
  }
});
```

Použití v testu

```
describe('User Registration with Faker.js', () => {
  it('should create a new user with random data', () => {
    cy.task('generateRandomUser').then((user) => {
      cy.log(`Generated User: ${user.username}, ${user.email}`);
    });
  });
});
```

Experimentální parametry Cypress v cypress.config.js

Cypress má několik experimentálních funkcí, které lze povolit v konfiguraci.

Běžné experimentální vlastnosti

1. `experimentalSessionAndOrigin` (Sleduje uživatelské relace napříč testy)

2. `experimentalModifyObstructiveThirdPartyCode` (Ošetřuje třetími stranami vkládané skripty)
3. `experimentalWebKitSupport` (Aktivuje testování v prohlížeči WebKit)
4. `experimentalStudio` (Automaticky zaznamenává a generuje testovací kód)

Příklad konfiguračního souboru s experimentálními vlastnostmi

```
module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    experimentalSessionAndOrigin: true, // Sleduje relace napříč testy
    experimentalModifyObstructiveThirdPartyCode: true, // Ošetření třetích stran
    experimentalWebKitSupport: true, // Testování v prohlížeči WebKit
    experimentalStudio: true, // Aktivace Cypress Studio pro generování testů

    setupNodeEvents(on, config) {
      on('task', {
        logMessage(message) {
          console.log('LOG:', message);
          return null;
        }
      });

      return config;
    }
  }
});
```

C. Osvědčené postupy pro konfiguraci a proměnné prostředí

1. Udržujte konfigurace přehledné a bezpečné:

- **Centralizace:**

Uchovávejte všechna globální nastavení na jednom místě (např. `cypress.config.js`) pro snadnější údržbu.

- **Oddělení citlivých údajů:**

Uchovávejte citlivá data, jako jsou API klíče, v proměnných prostředí namísto přímého vložení do testovacích souborů.

- **Správa verzí:**

Konfigurační soubory by měly být ve správě verzí, ale pro citlivá data využívejte mechanismy specifické pro prostředí (např. tajné hodnoty v CI/CD).

2. Vyhněte se pevnému vložení hodnot:

- **Používejte proměnné prostředí:**

Nahraďte pevně vložené URL, tokeny a další konstanty proměnnými prostředí.

- **Znovupoužitelné proměnné:**

Využívejte proměnné v konfiguračním souboru, aby změny stačilo provést pouze na jednom místě.

- **Příklad:**

```
// Vyhněte se pevnému vložení:  
cy.visit('http://localhost:3000/login');  
  
// Místo toho použijte:  
cy.visit('/');  
// baseUrl je definován v konfiguračním souboru.
```

3. Dokumentace a komentáře:

- Dokumentujte nastavení a proměnné prostředí, aby ostatní členové týmu rozuměli jejich účelu a použití.
- Veďte jasné komentáře v souboru `cypress.config.js` k vysvětlení jednotlivých nastavení.

D. Praktické aktivity

1. Konfigurace Cypress pomocí `cypress.config.js`:

- **Aktivita:**

Projděte si nastavení základního souboru `cypress.config.js`.

- **Úkol:**

Nakonfigurujte `baseUrl`, `viewport` a výchozí `timeouty`. Ověřte vliv těchto změn na chování testů.

2. Zavedení proměnných prostředí:

- **Aktivita:**

Nastavte proměnné prostředí v konfiguračním souboru pro různá prostředí (vývoj, staging, produkce).

- **Úkol:**

Vytvořte podmíněné logiky pro přepínání `baseUrl` a `apiUrl` podle příznaku prostředí. Spusťte testy v různých režimech a zkontrolujte správné načtení nastavení.

3. Přístup k proměnným prostředí v testech:

- **Aktivita:**

Napište jednoduchý Cypress test, který načte data z API pomocí proměnné prostředí pro

URL.

- **Úkol:**

Použijte v testu `Cypress.env('apiUrl')` a logujte nebo ověřte odpověď.

4. Simulované přepínání prostředí:

- **Aktivita:**

Ukažte spuštění testů s různými konfiguracemi prostředí přes příkazovou řádku.

- **Úkol:**

Použijte příkaz jako `npx cypress run --env config=production` a ukažte, jak testy mění své chování dle prostředí.

E. Funkčnost a konfigurace příkazů Cypress

- **Použití příkazů Cypress:**

Představte, jak příkazy Cypress (např. `cy.visit()`, `cy.get()`) automaticky využívají konfigurace uvedené v `cypress.config.js`. Například `cy.visit('/')` použije `baseUrl` z konfigurace.

- **Integrace vlastních příkazů:**

Stručně uveďte, že je možné vytvářet vlastní příkazy, které také využívají proměnné prostředí a konfigurace, což vede k flexibilnějším a lépe udržovaným testům.

F. Diskuze a otázky & odpovědi

Možné otázky studentů:

1. **Q:** *Co když potřebuji často měnit základní URL (baseUrl)?*

A: Použijte `baseUrl` v souboru `cypress.config.js`. Centralizujete tak nastavení URL a všechny testy `cy.visit('/')` budou používat novou adresu automaticky.

2. **Q:** *Jak mohu spravovat citlivá data (např. API klíče) bez jejich pevného vložení?*

A: Uchovávejte citlivé informace jako proměnné prostředí v konfiguračním souboru nebo v externím zabezpečeném úložišti (např. CI/CD tajné proměnné). Přistupujte k těmto hodnotám v testech pomocí `Cypress.env()`.

3. **Q:** *Mohu použít různá nastavení konfigurace pro různé běhy testů?*

A: Ano, můžete předávat proměnné prostředí přes příkazovou řádku (např. `npx cypress run --env config=staging`) nebo použít podmíněnou logiku v konfiguraci ke načtení různých nastavení.

4. **Q:** *Proč je důležité mít konfigurační soubory ve správě verzí?*

A: Správa verzí udržuje konzistenci napříč týmem, usnadňuje sledování změn a zajišťuje společné používání konfigurací při spouštění testů.

5. **Q:** *Jak ověřím, že jsou proměnné prostředí správně načtené v testech?*

A: Můžete je logovat v testu pomocí `cy.log(Cypress.env('apiUrl'))` nebo je ověřit asercí na očekávanou hodnotu.

6. **Q:** *Proč potřebujeme `setupNodeEvents` v Cypress?*

A: `setupNodeEvents` je potřeba pro rozšíření funkčnosti Cypress. Umožňuje nám registrovat posluchače událostí, definovat vlastní úlohy a ovlivnit průběh běhu testů.

7. **Q:** *Jak `on('task')` zlepšuje testování?*

A: Umožňuje komunikaci mezi Cypress (v prohlížeči) a Node.js (na backendu). To je užitečné pro logování, operace se soubory, API komunikaci a generování dynamických testovacích dat.

8. **Q:** *Jak mohu použít proměnné prostředí uvnitř `setupNodeEvents` ?*

A: K proměnným prostředí můžete přistupovat pomocí `process.env` v konfiguračním souboru a předat je do `Cypress.env()` .

9. **Q:** *Co se stane, když úloha nevrátí žádnou hodnotu?*

A: Cypress požaduje, aby všechny úlohy vrátily hodnotu nebo explicitně `null` .

10. **Q:** *Jsou experimentální funkce v Cypress stabilní?*

A: Ne vždy. Tyto funkce jsou ve vývoji a mohou se měnit. Před použitím v produkci si vždy zkontrolujte dokumentaci Cypress.

11. **Q:** *Proč nastavujeme `baseUrl` ?*

A: Nastavení `baseUrl` umožňuje používat v testech relativní cesty (například `cy.visit('/login')`) místo pevně vložených URL. Testy jsou tak přehlednější a flexibilnější při změně prostředí.

12. **Q:** *Jaký je účel sekce `"env": {}` ?*

A: Sekce `"env": {}` ukládá proměnné prostředí, ke kterým lze v testech přistupovat přes `Cypress.env()` . Používá se pro externalizaci citlivých údajů a konfigurací, které se mohou mezi prostředími lišit (například API endpointy nebo přístupové údaje).

13. **Q:** *Jak `numTestsKeptInMemory` pomáhá při ladění?*

A: Tato vlastnost omezuje počet uložených snímků testů v paměti, čímž Cypress umožňuje časové ladění bez nadměrné spotřeby paměti.

14. **Q:** *K čemu slouží `watchForFileChanges` ?*

A: Nastavuje, jestli má Cypress automaticky znovu spouštět testy při změně souborů v projektu. Ve vývoji se používá na `true` , v CI obvykle na `false` pro omezení zbytečných běhů.

15. **Q:** *Proč používat vlastnost `retries` ?*

A: Vlastnost `retries` pomáhá zvládnout nestabilní testy automatickým opakováním před neúspěchem a snižuje tak falešně negativní výsledky způsobené přechodnými problémy.

16. **Q:** *Co jsou experimentální funkce v Cypress, jako `experimentalSessionAndOrigin` ?*

A: Experimentální funkce jsou nové možnosti, které Cypress testuje. Mohou zlepšit správu relací nebo chování třetích stran, ale nemusí být plně stabilní.

17. **Q:** *Jakou roli má `setupNodeEvents` a funkce `on('task')` ?*

A: `setupNodeEvents` umožňuje registrovat posluchače Node.js událostí a rozšiřovat funkčnost Cypress. `on('task')` umožňuje spouštět kód Node.js z testů, jako je logování nebo práce se soubory.

Dodatečné zdroje

- **Průvodce konfigurací Cypress:**

[Cypress Configuration Documentation](#)

- **Příklady použití proměnných prostředí:**

Najděte ukázkové projekty a tutoriály demonstrující použití proměnných prostředí v Cypress na GitHubu a oficiálním blogu Cypress.